

COL1000: Introduction to Programming

Lectures 22-24

**Keerti, Naveen, Madhukar, Venkata
Department of CSE, IIT Delhi**

So far in the course

Complex programs for solving computational problems

- Iteration
- Recursion
- Reason about termination, correctness, time complexity

Next few lectures

Programs for managing large amounts of data

- Emphasis on appropriate organisation of data
(and functions)

A motivating scenario: toy ERP system

I. Student data:

Large *database* managing the student's data

- **Entry-number**
- Name
- Email id
- Department
- Courses (semester wise) & Grades
- Semester-wise SGPA
- Overall CGPA

A motivating scenario: toy ERP system

II. Faculty data:

Large *database* managing the faculty's data

- **Employee-code**
- Name
- Email id
- Department
- Courses taught (semester wise)

A motivating scenario: toy ERP system

III. Courses data:

Large *database* managing the courses data

- **Course code** (eg: 2502COL1000)
- Instructor(s)
- Students registered & their grades
(student entry number, their final score, and grade)

A motivating scenario: toy ERP system

Want to support queries:

*What grade does faculty member **ABC** give (on average) around the Median?*

How many students take both MTL1006 and PHY1006?

*When student **XYZ** did COL1000, how many students got an A grade that semester?*

A motivating scenario: toy ERP system

In theory, all of this can be done
(using Lists and Dictionaries)

Very cumbersome!

Potential duplication!

*Suppose we want to **update** a student's score and grade.
Need to make this update at several places!*

Python allows handling such scenarios
via Object Oriented Programming (OOP)

In Python, everything is an *object* of some *class*

Every *object* has certain *attributes*,
and *functions* that can operate on those *attributes*

Python allows handling such scenarios via Object Oriented Programming (OOP)

In Python, everything is an *object* of some *class*

**Where have we seen Python
objects so far?**

Classes: General structure

```
class ClassName:  
  
    def __init__(self, parameters):  
        self.instance_variable = value # object-specific data  
        # Initialization code
```

Classes: General structure

```
class ClassName:
    class_variable = value # shared across all objects (optional)

    def __init__(self, parameters):
        self.instance_variable = value # object-specific data
        # Initialization code

    def method_name(self, other_parameters):
        # Code that uses self.instance_variable
        pass

    @classmethod
    def class_method(cls, ...):
        # Operates on class-level data
        pass

    @staticmethod
    def static_method(...):
        # Utility method — operates of neither self nor cls
        pass
```

Example 1 : Classes and Objects

```
class student:
```

```
# constructor
```

```
def __init__(self, name, entry_num, dept):
```

```
    self.name = name
```

```
    self.entry_num = entry_num
```

```
    self.dept = dept
```

```
    self.courses_registered = []
```

```
class faculty:
```

```
# constructor
```

```
def __init__(self, name, empcode, dept):
```

```
    self.name = name
```

```
    self.empcode = empcode
```

```
    self.dept = dept
```

```
    self.courses_taught = []
```

```
class course:
```

```
# constructor
```

```
def __init__(self, coursecode):
```

```
    self.coursecode = coursecode
```

```
    self.instructor = []
```

```
    self.students_registered = {}
```

Class definition provides 'blueprint' for the objects

*Every object of class **student** has the following attributes:*

- name
- entry_num
- dept
- courses_registered

*Every object of class **faculty** has the following attributes:*

- name
- empcode
- dept
- courses_taught

Example 1 : Classes and Objects

```
class student:
```

```
# constructor
```

```
def __init__(self, name, entry_num, dept):
```

```
    self.name = name
```

```
    self.entry_num = entry_num
```

```
    self.dept = dept
```

```
    self.courses_registered = []
```

```
class faculty:
```

```
# constructor
```

```
def __init__(self, name, empcode, dept):
```

```
    self.name = name
```

```
    self.empcode = empcode
```

```
    self.dept = dept
```

```
    self.courses_taught = []
```

```
class course:
```

```
# constructor
```

```
def __init__(self, coursecode):
```

```
    self.coursecode = coursecode
```

```
    self.instructor = []
```

```
    self.students_registered = {}
```

```
student1 = student("Gauss", "2024AM1001",  
"AM")
```

```
student2 = student("Hilbert", "2024CS1002",  
"CS")
```

```
faculty1 = faculty("Euler", "20001", "CS")
```

```
faculty2 = faculty("Riemann", "20002", "AM")
```

```
course1 = course("2501COL100")
```

Objects are created by
class_name(parameters)

Object Oriented Programming (OOP)

OOP allows us to define our own data-types
Ties together data, and the methods that act on this data

Class: defines the template for the data
Specifies what attributes the objects of this class will have
And what methods can act on the objects of this class

Object: an instance of a class

Three important utilities of OOP

Abstraction

User does not need to know how the data is stored
Or how the methods are operating on the data

*Example: do we need to know how lists are stored?
And do we need to know how L.sort() works?*

Encapsulation

Ties together the data, and the methods that act on this data

Example: L.sort() acts only on the list L

Inheritance

Can create sub-classes that 'inherit' properties from parent class

Example of abstraction and encapsulation

Goal: Define a data-type called 'matrix'.

Initialisation:

- As a list of lists. The ith list corresponds to ith row of matrix

Associated functions:

- `add_column( L )` : takes as input a list L, adds L as a column to the matrix
- `add_row( L )` : takes as input a list L, adds L as a row to the matrix
- `transpose()` : takes no input, computes transpose of the matrix
- `is_symmetric()` : returns True if the matrix is a square matrix, and is symmetric. Else returns False

User does not need to know how this class is defined. The user only needs to know how to initialise an object of this data-type/class, and what operations can be performed on the objects of this data-type/class.

Example of abstraction and encapsulation

Goal: Define a data-type called 'matrix'.

Initialisation:

- As a list of lists. The ith list corresponds to ith row of matrix

```
# Example

M = matrix([[1, 2, 3], [4, 2, 1]])

M.add_column([2, 4])
M.add_row([2, 3, 1, 4])

M.transpose()

print(M.is_symmetric())
```

The matrix class

```
class matrix:

    #constructor.
    # Assuming L is a nonempty list of lists
    # each of the lists have equal length
    def __init__(self, L):

        # storing the number of rows and number of columns
        self.n = len(L)
        self.m = len(L[0])

        #initializing the matrix as an empty list
        self.M = []

        # we are creating a copy of the input lists
        # so that later edits to L do not affect our object
        for l in L:
            copy_list = l.copy()
            self.M.append(copy_list)
```

Adding methods to the matrix class

```
# method to add a row to the matrix
def add_row(self, L):

    # we will ensure that the new row has
    # the same length as all other rows

    if(len(L) != self.m):
        return None

    # creating a copy of the row
    # append that copy to the matrix
    copy_list = L.copy()
    self.M.append(copy_list)

    # increment the number of rows
    self.n = self.n + 1
    return None
```

Adding methods to the matrix class

```
# method to add a column to the matrix
def add_column(self, L):

    # we will ensure that the new column has
    # the same length as all other columns

    if(len(L) != self.n):
        return None

    # appending the ith element of L
    # at the end of the ith row
    # self.M[i] is the ith row
    for i in range(len(L)):
        self.M[i].append(L[i])

    # increment the number of columns
    self.m = self.m + 1
    return None
```

Adding methods to the matrix class

```
# method to check if a matrix is symmetrix
def is_symmetric(self):

    # if number of rows != number of columns
    # then return False
    if(self.n != self.m):
        return False

    # go over each element one by one
    # check that (i,j) entry is equal to (j,i) entry
    for i in range(self.n):
        for j in range(self.m):
            if(self.M[i][j] != self.M[j][i]):
                return False

    return True
```

Adding methods to the matrix class

```
def transpose(self):  
    # swap the values of n and m  
    self.n, self.m = self.m, self.n  
  
    # copy of M is needed for computing the transpose  
    copy_M = self.M  
  
    self.M = []  
  
    for i in range(self.n):  
        # ith iteration computes ith row of transpose  
        self.M.append([])  
  
        # next, we add elements to ith row, one by one  
        for j in range(self.m):  
            self.M[i].append(copy_M[j][i])
```

Adding methods to the matrix class

```
# method to print the matrix
def print(self):
    for i in range(self.n):
        for j in range(self.m):
            print(self.M[i][j], end=" ")
        print()
```

Calling methods of an object

```
M = matrix([[1, 2, 3], [4, 2, 1]])
```

```
M.add_column([2, 4])
```

```
M.add_row([2, 3, 1, 4])
```

```
print("Matrix after adding row and column:")
```

```
M.print()
```

```
print("Matrix after transpose:")
```

```
M.transpose()
```

```
M.print()
```

```
print("Is M symmetric?", M.is_symmetric())
```

Special methods : magic methods

How to check if two matrices are identical?

Here, we are interested in the entries of the matrices,
not the objects themselves

Approach 1

Define a method `is_equal(self, object)`

`M.is_equal(N)` : checks if the entries of M and N are equal

Approach 2

`M == N`

To make this approach work, we need to define `__eq__`

Special methods : magic methods

Using magic methods, one can provide class-specific definitions for infix operations

```
+ : __add__  
- : __sub__  
* : __mul__  
== : __eq__  
<= : __le__  
>= : __ge__  
etc
```

Can also define len() and str() for your own class using `__len__` and `__str__` respectively

Exercise: Polynomial class

Create a class for polynomials.

The constructor should take a list L of coefficients to initialise the polynomial.

If L is empty, then it corresponds to the zero polynomial, whose degree is set to `None`.

Methods:

$P.\text{deg}()$: returns the degree of P

$P.\text{eval}(x)$: takes as input x , and returns polynomial evaluation at x

$P.\text{add}(Q)$: updates P to $P+Q$

Exercise: Polynomial class

```
class Polynomial:
    def __init__(self, L):
        self.coeffs = L.copy()
        if(len(L) == 0):
            self.deg = None
        else
            self.deg = len(L) - 1
```

If $L = [2, 3, 4]$, then this represents the polynomial

$$2 + 3x + 4x^2$$

Any issues in the above code?

What if $L = [2, 3, 0]$?

Exercise: Polynomial class

```
class Polynomial:
    def __init__(self, L):
        self.coeffs = L.copy()
        self.trim()

    def trim(self):
        tmp = self.coeffs.copy()

        for i in range(len(self.coeffs)-1, 0, -1):
            if(self.coeffs[i] != 0):
                break
            tmp.pop()

        self.coeffs = tmp

        if(len(self.coeffs) == 0):
            self.deg = None
        else:
            self.deg = len(self.coeffs) - 1
```

Any issues in this code?

What if $L = [0]$?

Exercise: Polynomial class

```
class Polynomial:
```

```
    def __init__(self, L):
```

```
        self.coefs = L.copy()
```

```
        self.trim()
```

```
    def trim(self):
```

```
        tmp = self.coefs.copy()
```

```
        for i in range(len(self.coefs)-1, -1, -1):
```

```
            if(self.coefs[i] != 0):
```

```
                break
```

```
            tmp.pop()
```

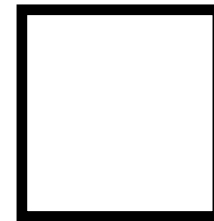
```
        self.coefs = tmp
```

```
        if(len(self.coefs) == 0):
```

```
            self.deg = None
```

```
        else:
```

```
            self.deg = len(self.coefs) - 1
```



Exercise: Polynomial class

```
def deg(self):  
    return self.deg
```

Exercise: Polynomial class

```
def add(self, Q):
```

Need to add
Q to self

```
.....
```

What if
 $Q.\text{deg} > \text{self}.\text{deg}$?

Are we done?

```
.. ..
```

Exercise: Polynomial class

```
def eval(self, x):
```

What is the
time complexity
of this function?

$O(deg^2)$

Can we improve this?

Exercise: Polynomial class

$O(deg)$ implementation using **Horner's method**

$$\begin{aligned}\text{Suppose } p(x) &= p_0 + p_1x + p_2x^2 + \cdots + p_kx^k \\ &= (\dots ((p_kx + p_{k-1})x + p_{k-2})x + p_{k-3}) \dots)x + p_0\end{aligned}$$

```
def linear_eval(self, x):
```

```
    if(len(self.coeffs) == 1):  
        return self.coeffs[0]
```

```
    ans = self.coeffs[-1]*x + self.coeffs[-2]
```

```
    for i in range(self.deg-2, -1, -1):  
        ans = ans*x + self.coeffs[i]
```

```
    return ans
```

Exercise: Polynomial class

What is the output of the following code:

```
P = Polynomial([2, 3, 1, 4, 0])
```

```
Q = Polynomial([2, 3, 1, 4])
```

```
print(P == Q)
```

```
print(P.coeffs == Q.coeffs)
```

```
print(P.coeffs is Q.coeffs)
```

False

True

False

We can define `==` for polynomials
by adding `__eq__` method to the `Polynomials` class

Exercise: Polynomial class

```
def __eq__(self, Q):  
    if(self.deg != Q.deg):  
        return False  
  
    if(self.deg is None):  
        return True  
  
    for i in range(self.deg+1):  
        if(self.coeffs[i] != Q.coeffs[i]):  
            return False  
  
    return True
```

```
P = Polynomial([2, 3, 1, 4, 0])  
Q = Polynomial([2, 3, 1, 4])
```

```
print(P == Q)  
print(P is Q)
```

True
False

Exercise: Polynomial class

```
P = Polynomial([2, 3, 1, 4, 0])
```

```
Q = Polynomial([2, 3, 1, 4])
```

```
print(str(P))
```

```
print(P)
```

What is the output?

```
<__main__.Polynomial object at 0x100939fa0>
```

```
<__main__.Polynomial object at 0x100939fa0>
```

If `__str__` is defined, then `str(P)` will output accordingly

Creating your own data-types using OOP

```
P = Polynomial([2, 3, 1, 4, 0])
```

```
Q = Polynomial([2, 3, 1, 4])
```

```
print(type(P))
```

```
<class '__main__.Polynomial'>
```

Create a data type for Int_set. The set is initialised to be empty.

Methods:

S.insert(x) : should insert x into set, return nothing. If x is already in the set, do nothing.

S.remove(x) : should remove x from the set (and print Error if x is not in set)

S.is_in(x) : returns True if x is in the set, else returns False

union(S,T) : returns the union of S and T

intersection(S, T) : returns the intersection of S and T

union and intersection are
not 'tied to' any object

‘Toy ERP’ system in more detail

A motivating scenario: toy ERP system

Large *database* managing the student's data, faculty data, courses

I. Student data:

- **Entry-number**
- Name
- Department
- Courses (*objects*)
- Overall CGPA

II. Faculty data:

- **Employee-code**
- Name
- Department

III. Courses data:

- **Course code** (eg: 2502COL1000)
- Instructor
- Students registered & their grades

```
class Student:
```

```
    count = 1000
```

```
    def _init_(self, name, year, dept):
```

```
        Student.count += 1
```

```
        self.name = name
```

```
        self.dept = dept
```

```
        self.entrynum = str(year) + str(dept) + str(Student.count)
```

```
        self.courses = []
```

```
        self.cgpa = 0.0
```

A motivating scenario: toy ERP system

Large *database* managing the student's data, faculty data, courses

I. Student data:

- **Entry-number**
- Name
- Department
- Courses (*objects*)
- Overall CGPA

II. Faculty data:

- **Employee-code**
- Name
- Department

III. Courses data:

- **Course code** (eg: 2502COL1000)
- Instructor
- Students registered & their grades

```
class Faculty:
```

```
    count = 1000
```

```
    def __init__(self, name, dept):
```

```
        Faculty.count += 1
```

```
        self.empcode = str(Faculty.count)
```

```
        self.name = name
```

```
        self.dept = dept
```

A motivating scenario: toy ERP system

Large *database* managing the student's data, faculty data, courses

I. Student data:

- **Entry-number**
- Name
- Department
- Courses (*objects*)
- Overall CGPA

II. Faculty data:

- **Employee-code**
- Name
- Department

III. Courses data:

- **Course code** (eg: 2502COL1000)
- Instructor
- Students registered & their grades
dict entrynum -> [marks, grade]

class Course:

```
def __init__(self, year, semester, coursename, faculty):
    self.coursecode = str(year)+"0"+str(semester)+str(coursename)
    self.instructor = faculty
    self.registered_students = {}
    # dict stores entrynum -> [marks, grade]
```

```
def add_student(self, student_obj):
    self.registered_students[student_obj.entrynum] = [None, None]
    student_obj.courses.append(self)
```

A motivating scenario: toy ERP system

```
class Student:
```

```
    count = 1000
```

```
    def __init__(self, name, year, dept):
```

```
        Student.count += 1
```

```
        self.name = name
```

```
        self.dept = dept
```

```
        self.entrynum = str(year) + str(dept) + str(Student.count)
```

```
        self.courses = [] # Stores Course objects
```

```
        self.cgpa = 0.0
```

```
    def compute_cgpa(self):
```

```
        total_points = 0 # each point lies in {0,1,2,...,9,10}
```

```
        graded_count = 0
```

```
        for course_obj in self.courses:
```

```
            # record is [marks, grade]
```

```
            record = course_obj.registered_students[self.entrynum]
```

```
            if record[1] is not None:
```

```
                total_points += record[1]
```

```
                graded_count += 1
```

```
        if graded_count > 0: self.cgpa = float(total_points) / graded_count
```

```
        else: self.cgpa = 0.0
```

A motivating scenario: toy ERP system

```
class Course:
```

```
    def __init__(self, year, semester, course_name, faculty):
```

```
        self.coursecode = str(year) + "0" + str(semester) + str(course_name)
```

```
        self.instructor = faculty
```

```
        self.registered_students = {} # entrynum -> [marks, grade]
```

```
    def add_student(self, student_obj):
```

```
        self.registered_students[student_obj.entrynum] = [None, None]
```

```
        student_obj.courses.append(self)
```

```
    def assign_marks_grade(self, entrynum, marks, grade):
```

```
        if entrynum in self.registered_students:
```

```
            self.registered_students[entrynum][0] = marks
```

```
            self.registered_students[entrynum][1] = grade
```

A motivating scenario: toy ERP system

```
f1 = Faculty("Dr. Alex", "CSE")
f2 = Faculty("Dr. Bob", "MTH")
s1 = Student("Cynthia", 2023, "CSE")
col1000 = Course(2023, 1, "COL1000", f1)
mtl1000 = Course(2023, 1, "MTL1000", f2)

col1000.add_student(s1)
mtl1000.add_student(s1)

col1000.assign_marks_grade(s1.entrynum, 90, 10)
mtl1000.assign_marks_grade(s1.entrynum, 70, 8)

s1.compute_cgpa()

print("Student:", s1.name, s1.entrynum)
print("COL1000 marks, grade:", str(col1000.registered_students[s1.entrynum]))
print("Final CGPA: " + str(s1.cgpa))
```

Inheritance

Suppose we have defined the class matrix

And now want to define a special sub-class called square_matrix
In addition to the attributes and methods of matrix, this sub-class can have its own attributes and methods

*For example, a square matrix has a diagonal
And we can compute the transpose more efficiently*

Naive approach: copy the entire code for class matrix
Add extra attributes and methods that are relevant for square_matrix

OOP approach: square_matrix can 'inherit' the class matrix

The square_matrix class

```
class square_matrix(matrix):  
  
    def __init__(self, L):  
        super().__init__(L)  
        self.diag = []  
        for i in range(self.n):  
            self.diag.append(self.M[i][i])
```

Note: square_matrix class inherits the class matrix

Can define and re-define methods for subclass

```
def __len__(self):  
    return self.n
```

```
def transpose(self):  
    for i in range(self.n):  
        for j in range(i+1, self.n):  
            self.M[i][j], self.M[j][i] = self.M[j][i], self.M[i][j]
```

```
M = square_matrix([[1,2], [3,4]])  
print("Printing the square matrix: ")  
M.print()  
print(len(M))  
print("diagonal of the matrix: ", M.diag)  
M.transpose()  
print("Printing the square matrix after transpose : ")  
M.print()
```